COMP-1832 Coursework

Python Part

Task-1:

Adjacency Matrix showing connectivity of all the stations:

	Baker Street	Bond Street	Covent Garden	Euston	Goodge Street	Green Park	Holborn	King's Cross St Pancras	Lancaster Gate	Leicester Square	Marble Arch	Notting Hill Gate	Oxford Circus	Piccadilly Circus	Queensway	Tottenham Court Road	Warren Street	Waterloo	Westminster
Baker Street	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bond Street	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0
Covent Garden	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0
Euston	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0
Goodge Street	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
Green Park	0	1	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	1
Holborn	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
King's Cross St Pancras	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Lancaster Gate	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0
Leicester Square	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0
Marble Arch	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Notting Hill Gate	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
Oxford Circus	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0
Piccadilly Circus	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0
Queensway	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0
Tottenham Court Road	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0
Warren Street	0	0	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0
Waterloo	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
Westminster	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 1: Adjacency Matrix

Task-2:

Two aspects of the plot that need improvements are:

1. Several stations—such as Green Park, Bond Street, Euston, Leicester Square, and Warren Street appear on multiple tube lines, meaning they are interchange stations. Interchanges are crucial because they allow travelers to switch lines and find the shortest and most efficient routes. The map has bold labels for these stations but they are not visually distinctive and clear indication as interchanges, travelers may overlook these key transfer points and choose unnecessarily long routes because of this. For example: A traveler going from Waterloo to Piccadilly Circus might miss that Green Park is a major 3-line interchange. As a result, they could take multiple unnecessary lines instead of the simple and efficient route: Jubilee Line to Green Park -> interchange to the Piccadilly Line -> Piccadilly Circus.

2. The second issue is that all edges are drawn in the same grey color and each station node represents an individual tube color. This is misleading because many stations belong to multiple tube lines, and node color alone cannot show these alternative connections. Without distinguishing edges by line, the map cannot show valid travel paths, potentially misleading users on which line to take. One additional issue is that for a station like Euston, both Victoria and Northern lines link to the same Warrent Street station which is a case of two tube lines linked to the same station represented by a slight darker grey edge, but again no significant visual distinction to represent which tube lines and unclear indication of concurrency.

Task-3:

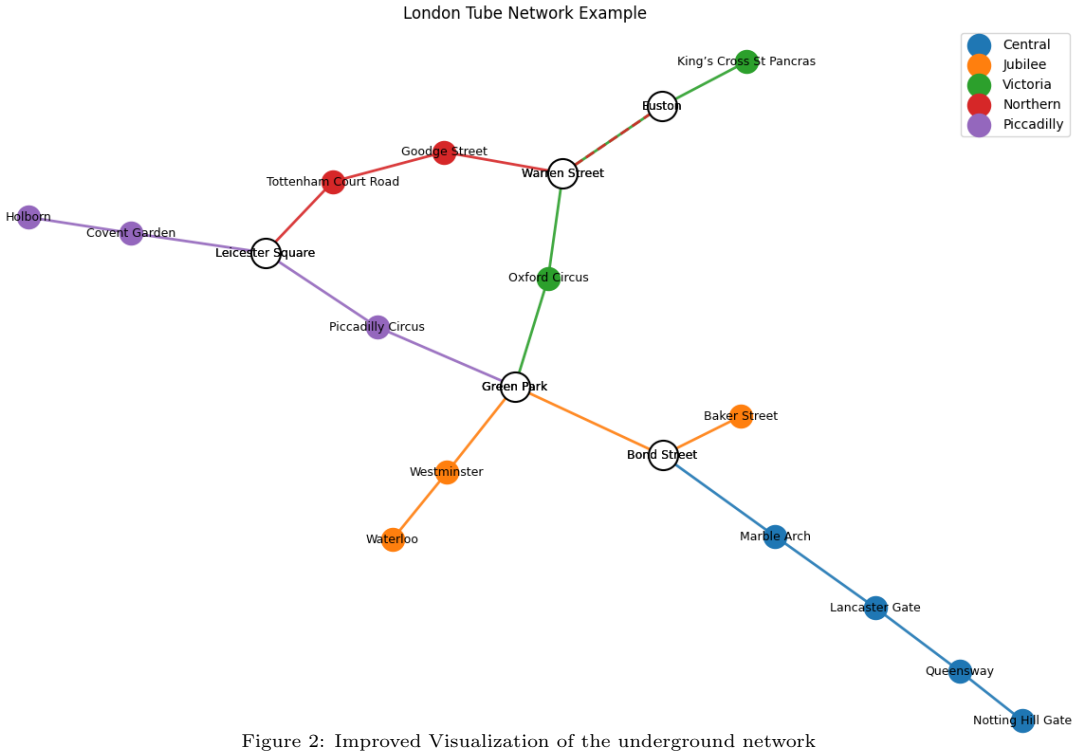

Figure 2: Improved Visualization of the underground network

Plot Interpretation: In the new map, the interchange stations are distinct(white halo-circles) and each colored edge now represents line's connectivity between stations, for Euston to Warren Street there is a red with green dashed line indicating both lines that exist between them.

Code Revisions:

The code from **L35** to **L37** will be corrected as:

```
edge_seen = {} #Dictionary to track occurrence of concurrent lines
station_count = {} #Dictionary to track stations count traversed by a line
for line, stations in lines.items():
    for s in stations:
        station_count[s] = station_count.get(s, 0) + 1 #Count how many times each station appears
            across all line
    interchanges = [s for s, c in station_count.items() if c > 1] #Logic to identify interchange
        stations if stations appear in more than 1 tube line
```

The code from **L40** to **L43** will be corrected as:

```
for i in range(len(stations)-1):
        key = tuple(sorted((stations[i], stations[i+1])))
        edge_seen[key] = edge_seen.get(key, 0) + 1 #Counting edge appearances to detect concurrent
            lines from one station
        style = "solid" if edge_seen[key] == 1 else "dashed" #Solid if there exist only a single
            line between stations, dashed if multiple lines
        nx.draw_networkx_edges(G, pos, edgelist=[(stations[i], stations[i+1])], width=2, alpha=0.9,
            edge_color=[plt.cm.tab10(line_index)], style=style) #Coloring the edges to represent
            tube lines and increased alpha for transparency
```

Between **L48** to **L50**, the code will be added as:

```
#Logic to draw interchange stations as Halo-circles
    nx.draw_networkx_nodes(G, pos, nodelist=interchanges,node_color="white", node_size=500,
        edgecolors="black", linewidths=1.5)
```

R Part

Task-1:

The incorrect visualization is due to the mismatch between the data structure and the statistical function of the Box Plot. The provided code loads the data frame `df_banking_long` with each row of total count values for each Year. The `geom_boxplot()` function is designed to visualize the distribution of sample (n>1) by computing and displaying its first quartile (Q1), median, third quartile (Q3), and range, but when it receives only one value per year, Q1, median, and Q3 are all calculated from a single value. So, the resulting box collapses into single flat lines for each year. This fails to show any meaningful dispersion. Additionally, `geom_jitter()` adds those single points directly on top of these lines.

Issues with the provided codes are: To read excel files, `readxl` package is not installed, and `tidyr` package is not loaded for data manipulation. Filename specified in file path is incorrect, there is no proper dataframe slicing and preprocessing for rows and columns. Provided column names don't exist.

New plot:

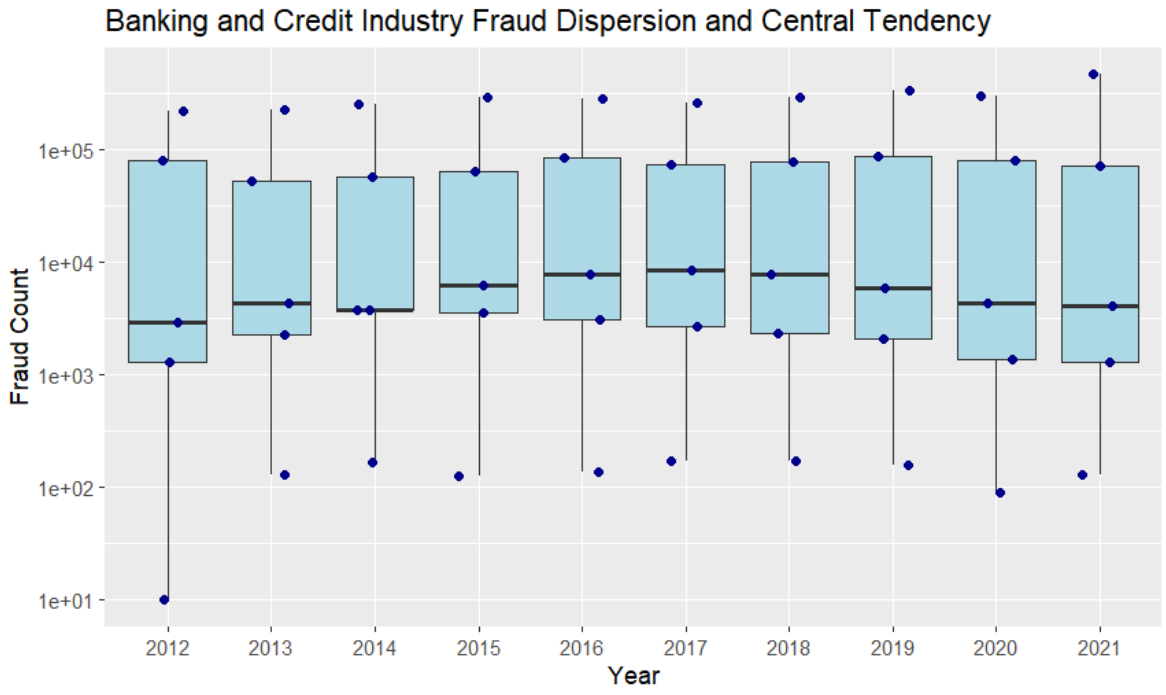

Figure 3: Updated plot of Banking and Credit Industry Fraud Dispersion and Central Tendency

Corrections in the code:
The code from **L86** to **L90** will be corrected as:

```
install.packages("readxl") #For interacting with excel files
library(readxl)
library(tidyr) #For data manipulation
file_path<- "Data\\cw_r.xlsx" #correct file name
df_banking<-read_excel(file_path, sheet =‘Table 3d‘, skip=8, n_max = 6) %>% slice(2:6)%>% select
    (1:11) #Sheet name correction and reading the necessary rows and columns for analysis from that
    sheet. Instead of single total rows for each year, slicing to include sub-category rows of
    Banking and credit industry fraud per year
colnames(df_banking)<-c(‘Banking and credit industry Frauds‘,"2012","2013","2014","2015","2016","
    2017","2018","2019","2020","2021") #Renaming to proper column names
```

Between **L98** and **L99**, the code will be added as:

```
scale_y_log10()+ #Log scaling fraud count data as it is highly skewed (different fraud categories
    have vastly different numerical ranges), which compresses the rest of the distribution.
```

Task-2:

The wrong visualization is because the solution code does not filter for regions of England only, instead it also includes section of WALES. The grouping is done on the wrong column without filtering on the proper area code. Unwanted strings also appear on the plot. Other issues with provided code are: The `groupby()` logic tries grouping on individual area codes resulting in 40+ groups instead of required nine regions of England due to missing area code filtering logic and no string handling in the data.

New plot:

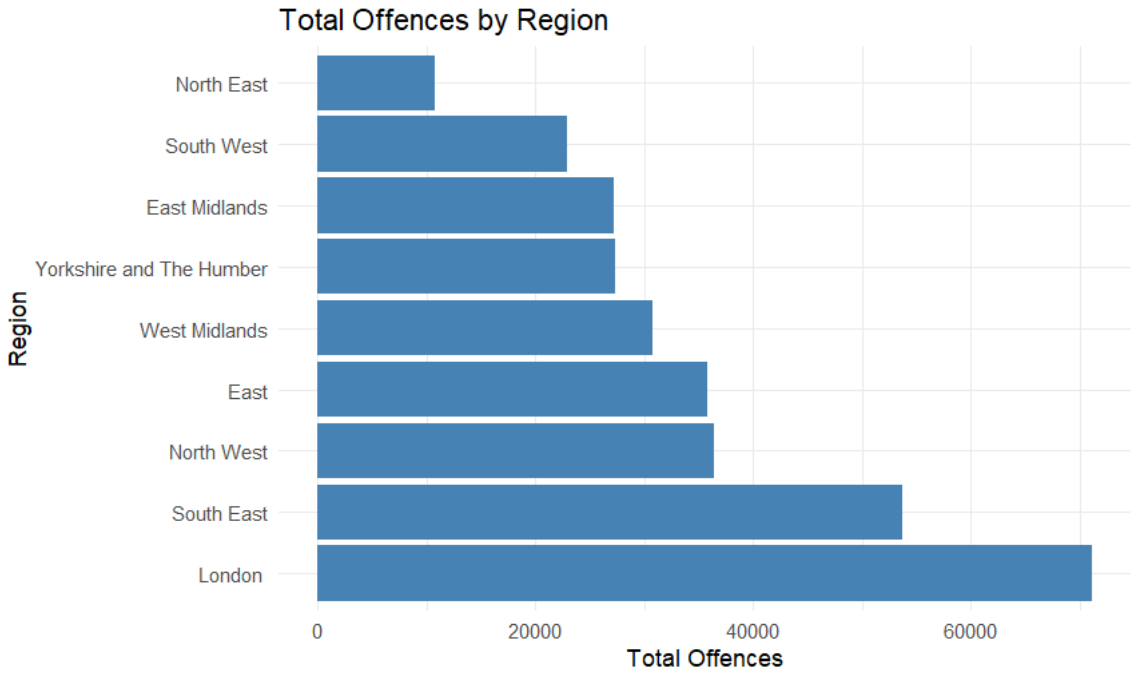

Figure 4: Updated plot for Total count of offence in England regions

Corrections in the code:
The code from **L134** to **L136** will be revised as:

```
colnames(data_table5)<-c("Area_Code","Region","Total_Offences","Offence_Rate_per_1000", "Percentage_
    Change") #Proper columns renaming for area codes and region
```

The code from **L141** to **L142** will be revised as:

```
#Additional filtering logic for only the England regions i.e Area_Code starting with "E12"
filter((!is.na(Region) & !grepl("Link|Source", Region) & grepl("^E12", Area_Code))) %>% mutate(Total
    _Offences
```

Between **L144** and **L145**, the code will be added as:

```
,Region = gsub("\\[note\\s\\d+\\]", "", Region)) #logic to remove unnecessary strings like London
    has "note[14]" string attached in data
```

Plot Interpretation:

1. **London** has the highest fraud offences by a large margin, as it records 71,203 offences, exceeding all other regions. This suggests that the capital remains the primary hotspot for fraud related crimes due to its economic prosperity.
2. The **North East** stands out with the lowest number of offences, reporting only 10,718 offences. This might be due to its low population density, less commercialization, and dedicated regional police to tackle this crime.
3. The **South East** is second most, maybe because of its close proximity to **London**, but all other regions show considerable to moderate cases.

Declaration of AI Use

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – NO
- To create an outline of the topic – NO
- To explain concepts – YES
- To support my use of language – YES
- In other ways, as described below:
    1. Code simplification and alternative code syntax suggestions
    2. Plot comparisons